

Plan de Manejo y Documentación de Datos (DMP)

Proyecto de Tesis Doctoral: Diseño de un marco metodológico para la implementación de un Sistema de Vigilancia y alerta temprana para la Influenza Aviar bajo el enfoque Una Salud.

Programa: Doctorado en Ingeniería y Ciencias con la Industria (DICI)

Estudiante: Jorge Eduardo Taucare Miranda

Fuentes de Datos y Unidad de Análisis

El proyecto procesa información transdisciplinaria bajo el enfoque integral "Una Salud", articulando tres tipos de fuentes de datos:

Fuentes Estructuradas (IBS): Registros epidemiológicos oficiales de brotes y mortalidad animal del Servicio Agrícola y Ganadero (SAG), informes de laboratorio clínicos, estadísticas de salud humana zoonótica del Ministerio de Salud y su Departamento de Estadísticas e Información de Salud (MINSAL/DEIS), censos y movimientos de aves, datos meteorológicos y climáticos de la Dirección Meteorológica de Chile (DMC), y datos filogenéticos del virus.

Fuentes No Estructuradas (EBS): Reportes comunitarios espontáneos, denuncias ciudadanas al SAG, prensa digital local, y menciones en redes sociales como alertas en X/Twitter.

Fuentes Automatizadas: Transmisiones de video e imágenes capturadas por cámaras fijas y drones en zonas prioritarias como humedales y planteles industriales, procesadas mediante arquitecturas de Deep Learning como la familia YOLO.

Unidad de Análisis: El evento epidemiológico espacio-temporalizado, agregado a nivel de coordenadas geográficas o código de comuna (CUT), estructurado por semanas epidemiológicas.

Procedimientos de Almacenamiento y Respaldo

Infraestructura de Almacenamiento: Los archivos crudos (data_raw/) se mantendrán almacenados de forma local en la estación de trabajo dedicada al procesamiento de visión computacional y se sincronizarán mediante almacenamiento seguro en la nube institucional de la universidad a través de [OneDrive](#) o SharePoint privado.

Estrategia de Respaldo: Se aplicará una política de redundancia automatizada. Las copias de seguridad del código y las bases de datos procesadas se gestionarán a través de este repositorio en GitHub. Los archivos de datos voluminosos que superen los 100 MB, como videos de drones, cámaras o imágenes de entrenamiento, se omitirán del control de versiones directo mediante la configuración de un archivo .gitignore, o se gestionarán con Git LFS (Large File Storage).

Consideraciones de Confidencialidad y Acceso

Privacidad de la Información: Para dar cumplimiento a la Ley Número 19.628 de protección de la vida privada en Chile, todos los datos vinculados a planteles comerciales específicos, criaderos de traspatio o fichas clínicas de pacientes serán estrictamente anonimizados.

Agregación Territorial: Las coordenadas geoespaciales sensibles de predios privados se someterán a un proceso de agregación espacial utilizando el Código Único Territorial (CUT) provincial o comunal, impidiendo la identificación directa de actores del sector productivo.

Acceso y Licenciamiento: El código fuente alojado en las carpetas /code se compartirá bajo una licencia de código abierto como MIT o Apache 2.0. Los datos procesados intermedios anonimizados (data_processed/) y las métricas

de los outputs se liberarán públicamente al finalizar la investigación doctoral para garantizar la transparencia y reproducibilidad de la Ciencia Abierta.

Estrategia de Documentación de Bases de Datos

Control de Versiones y Metadatos: Cada base de datos integrada en el sistema de vigilancia contará con una hoja de metadatos descriptiva compilada en el archivo diccionario_datos.xlsx alojado en la carpeta /docs.

Documentación del Código: Todos los scripts de Python y R destinados a la limpieza de datos, modelamiento causal (DAGs) y ejecución de la g-fórmula paramétrica incluirán un encabezado mandatorio con la estructura: Autor, Fecha de creación, Descripción del flujo de trabajo, Datos de Entrada (Inputs), Datos de Salida (Outputs) y Dependencias de Librerías.

Convenciones de Nombres de Archivos

Scripts de Código (/code): Se adoptará una estructura numérica secuencial que describa el flujo lógico de la tubería de datos (pipeline), utilizando minúsculas y guiones bajos:

01_data_extraction_ebs.py (Captura de alertas y NLP).
02_data_cleaning_ibs.R (Limpieza de registros oficiales SAG).
03_causal_dag_modeling.R (Construcción y análisis del DAG).
04_parametric_g_formula.R (Simulaciones contrafactuales).
05_computer_vision_yolo.py (Procesamiento de imágenes piloto).

Archivos de Datos: Se usará el formato YYYYMMDD_descripcion_fuente_version.csv como por ejemplo 20260325_sag_ibs_v1.csv.

Procedimientos para Garantizar la Calidad de los Datos

Validación Automatizada: Se programarán funciones específicas de aserción y reglas de validación lógica utilizando librerías especializadas como pydantic en Python o validate en R. Se inspeccionará de forma automatizada:

La integridad de los campos clave evitando identificadores duplicados.

Los límites geográficos de las comunas para que las coordenadas sean consistentes con el espacio soberano de Chile.

La consistencia cronológica asegurando que la fecha de notificación no sea previa a la fecha de ocurrencia estimada del síntoma o evento.

Manejo de Datos Faltantes: Se prohibirá estrictamente la alteración manual de registros en hojas de cálculo. Los valores perdidos se codificarán explícitamente como NA. De acuerdo con la estructura definida en el DAG, se emplearán técnicas de imputación múltiple o se evaluarán bajo los supuestos del análisis contrafactual, documentando el impacto de estas pérdidas en la sensibilidad del sistema.

Estrategias para Favorecer la Reproducibilidad

Gestión de Entornos Virtuales: Para erradicar conflictos de dependencias tecnológicas, el repositorio incluirá archivos de bloqueo de versiones. Para los scripts de inferencia causal en R se implementará renv generando el archivo renv.lock, y para los scripts de visión computacional en Python se adjuntará un archivo requirements.txt o environment.yml especificando las versiones exactas de frameworks como PyTorch y la arquitectura YOLO.

Pipelines de Datos Intactos: El flujo metodológico operará bajo el principio de inmutabilidad del dato crudo. Los archivos en data_raw/ nunca serán modificados; cualquier transformación, ingeniería de variables o unión de fuentes (IBS + EBS) se programará exclusivamente mediante código en la carpeta /code, permitiendo que cualquier evaluador replique los resultados de outputs/ de principio a fin.